LLM-Modulo Semantic-Numeric CEGIS

Learning constraints from expert trajectories and whole-trajectory safety feedback

What this document explains. The project learns a task constraint when the true constraint is hidden. It sees safe expert trajectories. It can also ask an external Oracle whether a new trajectory is safe. The Oracle gives one answer for the whole trajectory. It does not say which state caused a violation.

The system has two connected loops. An LLM searches over the *meaning and structure* of the constraint. Neural models learn the *numeric boundary*. A trajectory optimizer creates tests that may break the current models. The answers to these tests go back into both loops.

Main idea: the LLM decides what kind of constraint to test. Data and optimization decide where the boundary is. The LLM never guesses an obstacle center, radius, or a list of boundary points.

1 The learning problem

1.1 What the system can use

The training loop receives four things:

- a short task description d;
- a public variable schema $\mathcal{S}$, such as position and velocity;
- a set of expert trajectories $\mathcal{D}_E = \{\tau_i^E\}$ that are treated as safe;
- a trajectory membership Oracle $\mathcal{O}$.

A trajectory is a sequence of states,

$$\tau = (x_1, x_2, \ldots, x_T). \tag{1}$$

The Oracle returns only one label:

$$\mathcal{O}(\tau) \in \{0, 1\}, \qquad 0 = \textsc{safe}, \quad 1 = \textsc{violation}. \tag{2}$$

The training loop cannot read the true obstacle shape, state-level collision labels, the true violation time, or evaluation metrics such as IoU.

1.2 What the system must learn

The unknown constraint has two parts.

First, it has a structure:

$$H = (V, C, R, A, M). \tag{3}$$

V selects variables. C says how they are coupled. R gives the relation type. A gives the time rule. M selects a model family.

Second, it has numeric parameters ω. Together, H and ω define a state violation score and a trajectory violation score:

$$g_{H,\omega}(x_t) \in \mathbb{R}, \qquad G_{H,\omega}(\tau) = A_t\big(g_{H,\omega}(x_t)\big). \tag{4}$$

The sign has one fixed meaning throughout the code:

$$g \le 0 \text{ is safe}, \qquad g > 0 \text{ is a violation}. \tag{5}$$

2 Why the project uses two loops

A neural network can fit a boundary after its inputs and semantics are fixed. It cannot tell us whether the real constraint is about x position, speed, orientation, or a joint relation between several variables.

An LLM can use the task text to suggest these structures. But it should not estimate an exact numeric boundary from a few summary values. This leads to a clear split:

- **Semantic CEGIS:** search over discrete structures H.
- **Numeric CEGIS:** learn parameters ω_H and find counterexamples for each H.

The loops are connected. Numeric learning produces evidence. The LLM reads that evidence and changes the active hypotheses or the next tests. This is why the design is deeper than a one-time LLM call.

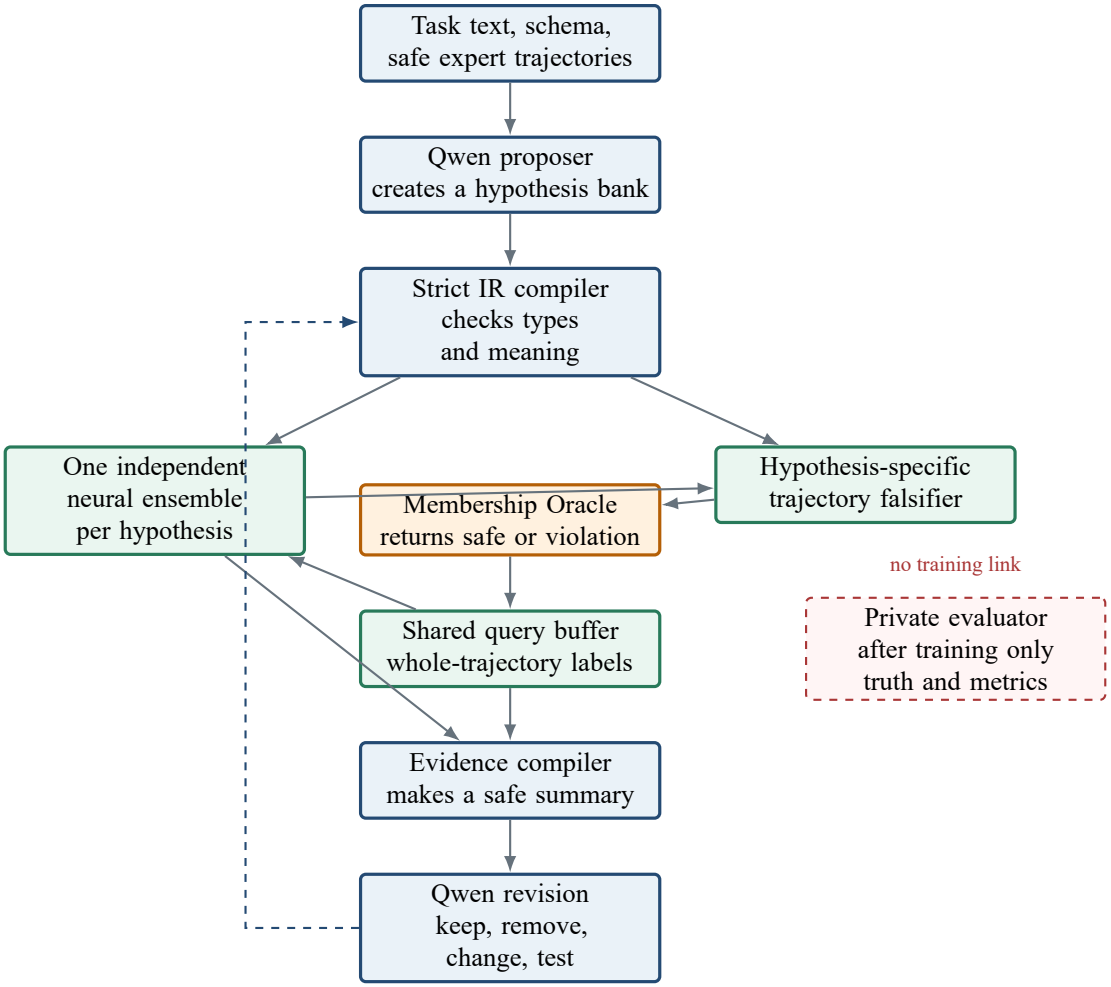

Figure 1: The full loop. The red dashed evaluator is separate from training.

3 The hypothesis language

3.1 A typed intermediate representation

The LLM does not return free-form Python code. It returns a small typed object:

```
H = {
  variables,          # observable features used by the constraint
  coupling,           # joint | independent
  relation,           # forbidden_region | upper_bound | lower_bound
  temporal_operator,  # max | mean | last
  model_family        # mlp | linear
}
```

The bank also stores an ID, a short reason, a risk direction, a parent ID, and a generation number. This gives a versioned set of hypotheses. The system is not forced to trust one first guess.

3.2 Each field changes real computation

Table 1: How semantic fields are compiled

Field	What the learner actually builds
`variables`	Selects input features and their normalization ranges.
`coupling=joint`	Sends all selected variables into one network. It can learn coupled shapes.
`coupling=independent`	Builds one network per variable and joins their violation scores with max, like an OR rule.
`forbidden_region`	Uses a learned zero level set $g(x) = 0$ as an implicit boundary.
`upper_bound` / `lower_bound`	Uses one learned scalar threshold. Only one variable is allowed.
`model_family=mlp`	Builds a nonlinear network.
`model_family=linear`	Builds a linear boundary for a simpler baseline.
`max` / `mean` / `last`	Changes how state scores become one trajectory score.

For a joint forbidden region, normalized features $z_t = N_V(x_t)$ enter one network:

$$g_{H,\omega}(x_t) = f_\omega(z_t). \tag{6}$$

For independent variables, the model builds one network per variable:

$$g_{H,\omega}(x_t) = \max_{j \in V} f_{\omega_j}(z_{t,j}). \tag{7}$$

For a scalar upper bound, the compiled form is

$$g_{\text{upper}}(x_t) = s(z_t - \theta), \qquad s = \text{softplus}(a) + 10^{-4} > 0. \tag{8}$$

The lower-bound form flips the sign. This means a simple speed limit can stay interpretable, while a two-dimensional obstacle can use a joint MLP.

3.3 The compiler is a safety gate

The compiler rejects unknown variables, unsupported values, duplicate active structures, and combinations that have no clear meaning. For example, a scalar upper bound cannot use two variables.

Qwen output may stop in the middle of JSON because of a token limit. The parser accepts only objects that are fully closed and pass all type checks. It never runs an incomplete object. If too few valid hypotheses remain, a fixed coverage bank adds basic alternatives. Every rejection and fallback is logged.

4 The numeric constraint learner

4.1 One model per hypothesis

Every active hypothesis owns its own neural ensemble. The models do not share a hidden feature that could leak the true obstacle. This matters because two hypotheses should compete on their own meaning.

For example:

- an x-only hypothesis sees only x;
- a speed hypothesis sees only speed;
- a joint (x, y) hypothesis sees both coordinates in one network;
- an independent (x, y) hypothesis uses two one-dimensional networks.

The ensemble members use different random seeds and bootstrap samples of Oracle queries. Their mean gives the model score. Their variance gives a simple estimate of model uncertainty.

4.2 From state scores to a trajectory score

Obstacle avoidance normally uses the rule "a trajectory fails if any state fails." The hard max is replaced during training by a smooth max:

$$G_\beta(\tau) = \frac{1}{\beta}\left[\log \sum_{t=1}^{T} \exp\big(\beta g(x_t)\big) - \log T\right]. \tag{9}$$

When β is large, this is close to $\max_t g(x_t)$. The subtraction of $\log T$ keeps the value normalized for the trajectory length.

The label rules are now simple:

$$\tau \text{ is safe} \Rightarrow g(x_t) \le 0 \text{ for all } t, \tag{10}$$

$$\tau \text{ is a violation} \Rightarrow g(x_t) > 0 \text{ for at least one } t. \tag{11}$$

This is multiple-instance learning. A trajectory is a bag. Its states are instances. A violation bag contains at least one violating instance, but the learner is not told which one.

4.3 Training losses

The code uses two soft margin losses:

$$\ell_{\text{SAFE}}(s) = \text{softplus}(s + m), \tag{12}$$

$$\ell_{\text{VIOLATION}}(s) = \text{softplus}(m - s), \tag{13}$$

where $m > 0$ is a margin. The first pushes a score below zero. The second pushes it above zero.

The full loss contains these parts:

$$\mathcal{L} = \lambda_E \ell_{\text{SAFE}}\big(G(\mathcal{D}_E)\big) + \lambda_{ES} \ell_{\text{SAFE}}\big(g(\mathcal{D}_E)\big) \tag{14}$$

$$+ \lambda_Q \ell_{\text{SAFE}}\big(G(\mathcal{D}_Q^{\text{SAFE}})\big) + \lambda_{QS} \ell_{\text{SAFE}}\big(g(\mathcal{D}_Q^{\text{SAFE}})\big) \tag{15}$$

$$+ \lambda_V \ell_{\text{VIOLATION}}\big(G(\mathcal{D}_Q^{\text{VIOLATION}})\big) + \lambda_W \ell_{\text{VIOLATION}}\big(g(\mathcal{W})\big) + \lambda_R \|g(\mathcal{D}_E)\|_2^2. \tag{16}$$

In plain language:

- every expert trajectory should be safe;
- every state in an expert trajectory should be safe;
- an Oracle-safe query should also be safe;
- an Oracle-violation query must contain a violation;
- likely violation states get an extra weak loss;
- a small regularizer keeps scores from growing without need.

4.4 How the learner handles an unknown violation time

The Oracle gives no local label. The system therefore estimates a small set of *latent witnesses*. For each state x_t in a violation trajectory, it measures distance from known safe states in normalized feature space:

$$d_t = \min_{y \in \mathcal{S}_{\text{SAFE}}} \|N_V(x_t) - N_V(y)\|_2 \,. \tag{17}$$

The most novel states become the witness set $\mathcal{W}$. The learner applies an extra violation loss to them.

A latent witness is only a weak guess. It is not a true collision label. It can be wrong. New counterexample queries are what correct this guess over time.

4.5 Ensemble uncertainty

For K ensemble members,

$$\bar{G}(\tau) = \frac{1}{K} \sum_{k=1}^{K} G_k(\tau), \qquad U(\tau) = \text{Var}_{k=1}^{K}\big[G_k(\tau)\big]. \tag{18}$$

Large $U(\tau)$ means the members disagree. The system uses this value when it chooses which hypotheses to trust and which trajectories to query.

5 The trajectory falsifier

The learner needs more than expert paths. Expert paths show safe behavior, but they do not tightly locate the unsafe boundary. The falsifier creates new paths that test a current hypothesis.

5.1 Warmup queries

At the start, the system mixes an expert path with the straight line from start to goal. It uses several mix strengths and small local deformations. These simple queries usually give an initial set of safe and violation labels.

5.2 Hypothesis-specific query modes

After warmup, each active hypothesis can request a different test:

- `model_false_safe`: find a path the model calls safe that the Oracle may reject;
- `model_false_unsafe`: find a path the model calls unsafe that the Oracle may accept;

- `boundary_uncertainty`: move close to $G = 0$ where ensemble members disagree;
- `shortcut`: shorten an expert detour and test why the expert avoided that area;
- `local_feature_stress`: change behavior linked to one selected variable.

5.3 Differentiable path optimization

The falsifier keeps the start and goal fixed. It optimizes only the middle waypoints. Its general objective is

$$\mathcal{J}(\tau) = \lambda_E d(\tau, \tau^E) + \lambda_S \,\text{Smooth}(\tau) + \lambda_L \,\text{Length}(\tau) \tag{19}$$

$$+ \lambda_B \phi_{\text{mode}}\big(\bar{G}(\tau)\big) - \lambda_U U(\tau) + \lambda_\Delta P_{\text{step}}(\tau) + \lambda_\Omega P_{\text{workspace}}(\tau). \tag{20}$$

The first terms keep the new path useful and physically reasonable. The semantic term pushes the path toward the requested failure mode. The negative uncertainty term prefers paths where ensemble members disagree. The last terms punish large steps and leaving the workspace.

For the three main modes,

$$\phi_{\text{false-safe}}(G) = \text{ReLU}(G + \epsilon)^2, \tag{21}$$

$$\phi_{\text{false-unsafe}}(G) = \text{ReLU}(\epsilon - G)^2, \tag{22}$$

$$\phi_{\text{boundary}}(G) = G^2. \tag{23}$$

Before a path reaches the Oracle, it must pass hard checks. Values must be finite. Start and goal must be unchanged. Every point must stay inside the workspace. Each step must stay below a limit.

6 Turning numeric results into LLM evidence

The Evidence Compiler is the only numeric-to-language bridge. It creates a short report for each active hypothesis. The report includes:

- accuracy on safe query trajectories;
- recall on violation trajectories;
- balanced accuracy;
- consistency with expert trajectories;
- false-safe and false-unsafe counts;
- mean score margin;
- ensemble uncertainty;
- the success rate of tests created for that hypothesis;
- a small structure complexity value.

The current ranking score is

$$S(H) = 0.42\,\text{BalancedAccuracy} + 0.18\,\text{ExpertSafeRate} \tag{24}$$

$$+ 0.15\,\text{ViolationRecall} + 0.10\,\text{SafeAccuracy} \tag{25}$$

$$+ 0.15\,\text{InterventionYield} - 0.015\,\text{Complexity} - 0.02\,\text{Uncertainty}. \tag{26}$$

This score is an engineering rule. It is not a theorem and it is not a calibrated probability. Its purpose is to make the evidence easy to compare and easy to audit.

7 What the LLM does inside the loop

The LLM first proposes a small population of structures. Later, it reads the evidence report and returns typed revision actions. Supported actions include:

- keep a hypothesis and query it again;
- remove a weak hypothesis;
- change its variables;
- change joint versus independent coupling;
- change the time operator;
- change linear versus MLP model family;
- split one hypothesis into several alternatives;
- add a new hypothesis;
- request a specific intervention.

Every action goes through the same strict compiler. The LLM cannot directly edit a neural network or insert hidden numeric values. If it returns one valid keep action and several invalid actions, the valid action can still be used. Invalid actions are logged and ignored.

If the LLM gives no valid champion, the system uses the highest numeric evidence score. This keeps the loop running even when a small local LLM makes a formatting mistake.

The LLM has a real control role: it can change the active model structures and the next tests. It does not train the boundary itself.

8 The complete loop, step by step

Step 1. Read the task text, variable schema, and safe expert trajectories.

Step 2. Ask Qwen for several constraint structures.

Step 3. Compile every structure. Reject invalid objects. Add coverage candidates if needed.

Step 4. Generate warmup trajectories and ask the Oracle for whole-trajectory labels.

Step 5. Build one independent neural ensemble for each active hypothesis.

Step 6. Train each ensemble with expert trajectories and all current Oracle queries.

Step 7. Use each hypothesis and its requested intervention to optimize new test trajectories.

Step 8. Validate each trajectory, query the Oracle, and add the result to the shared buffer.

Step 9. Retrain the active ensembles with the new labels.

Step 10. Compile a leakage-safe evidence report.

Step 11. Ask Qwen to keep, remove, replace, split, or test hypotheses.

Step 12. Apply only valid actions and start the next outer round.

Step 13. After all rounds are finished, select the champion and save all models and logs.

Step 14. Only then may a separate evaluator use private truth for plots or metrics.

The important point is the order. Private evaluation never changes training, ranking, or an LLM prompt.

9 Example: learning a two-dimensional obstacle

This is a simple running example. It shows how the modules work together.

Assume a point robot moves in a plane. Safe expert paths bend around an unknown obstacle. The public variables include x, y, v_x, v_y, and speed.

Initial semantic search. Qwen may propose an x bound, a y bound, a speed bound, a joint (x, y) forbidden region, and independent x/y forbidden bands. The compiler turns each option into a different model.

First numeric evidence. Warmup makes paths closer to the straight start-to-goal line. Some remain safe. Some violate the hidden rule. Every hypothesis trains on the same trajectory labels, but each sees only its own selected features.

Structure comparison. An x-only model may explain some unsafe paths but reject many safe detours. A speed model may fail because slow paths can still cross the obstacle. A joint (x, y) model can represent a local two-dimensional region and may fit both groups better.

Local boundary learning. The Oracle still does not point to a collision state. Smooth max says that at least one state in a violation trajectory must cross the learned zero level set. Latent witnesses give a weak first guess. False-safe, false-unsafe, and boundary queries then add more information near the current boundary.

Outer revision. The evidence report tells Qwen which structures explain safe paths, catch violation paths, and create useful tests. Qwen can keep the joint (x, y) model, remove weak one-dimensional models, or request more local queries.

Learned representation. The learned obstacle boundary is the zero level set

$$\{(x, y) : g_{H,\omega}(x, y) = 0\}. \tag{27}$$

This boundary comes from the neural model and Oracle evidence. It is not a circle proposed by the LLM.

10 Information separation

The benchmark Oracle may internally know a hidden obstacle so it can answer queries. The learner receives only a narrow membership view. It can call `query(trajectory)` and read the query count. It cannot call state collision functions or read the obstacle geometry.

Table 2: Who can see each type of information

Information	Learner	LLM	Evaluator	Purpose
Expert trajectories and schema	Yes	Definition	Yes	Public input
Whole-trajectory labels	Yes	Summary	Yes	Training evidence
Model scores and uncertainty	Yes	Summary	Yes	Model behavior
Obstacle geometry	No	No	Yes	Private evaluation only
State-level violation mask	No	No	Yes	Private evaluation only
Evaluation metrics	No	No	Yes	Final reporting only

The Evidence Compiler is not given the private evaluator. Its output explicitly states that no obstacle center, radius, state labels, or evaluation metrics are included.

11 What the current design can and cannot do

11.1 What it does well

- It separates semantic structure search from numeric boundary fitting.
- It uses real typed structures, not prompt text that has no effect on computation.
- It works with whole-trajectory labels.
- It can test several competing explanations at the same time.
- It creates targeted counterexamples instead of relying only on demonstrations.
- It records every LLM output, rejection, fallback, query, and bank update.
- It keeps private evaluation data outside the training loop.

11.2 What it does not guarantee

- A finite set of trajectory labels may not uniquely identify the true boundary.
- A latent witness may not be the true violation state.
- An MLP zero level set has no formal safety guarantee by itself.
- Qwen may fail to propose the best structure or may return invalid JSON.
- The evidence score is heuristic and may need tuning for a new task.
- Poor query coverage can produce a distorted boundary even when the structure is correct.

These limits are not hidden by the architecture. They are visible in uncertainty, false-safe and false-unsafe cases, expert consistency, query logs, and the surviving hypothesis bank.

12 Code map

Table 3: Main modules in the implementation

Module	Main job
`hypotheses.py`	Typed hypothesis IR, compiler, bank, and revision validation.
`semantic.py`	Qwen prompts, output parsing, partial JSON recovery, and fallback policy.
`data.py`	Public feature schema, differentiable derived features, and bounds.
`learner.py`	Constraint networks, temporal aggregation, ensembles, losses, and latent witnesses.
`falsifier.py`	Warmup paths, differentiable trajectory search, and validation.
`oracle.py`	Narrow trajectory membership interface and separate private evaluation access.
`evidence.py`	Leakage-safe model summaries and hypothesis ranking.
`loop.py`	The full semantic-numeric CEGIS controller and artifact saving.